

Empirical Comparison between MOEAs and Local Search on Multi-Objective Combinatorial Optimisation Problems

Miqing Li
m.li.8@bham.ac.uk
University of Birmingham
Birmingham, United Kingdom

Xiaochen Chu
chuxiaochen95@163.com
Harbin Institute of Technology
Harbin, China

Xiaofeng Han
hanx63520@gmail.com
Harbin Institute of Technology
Harbin, China

Zimin Liang
zxl525@student.bham.ac.uk
University of Birmingham
Birmingham, United Kingdom

ABSTRACT

Local search has gained its popularity in addressing multi-objective combinatorial optimisation problems (MOCOPs) within the communities of evolutionary computation and operational research. The ease of defining the neighbourhood in discrete spaces of MOCOPs makes local search well-suited to conducting step-wise moves. On the other side, evolutionary algorithms are amongst very first choices in solving various multi-objective optimisation problems. Although most multi-objective evolutionary algorithms (MOEAs) are developed, tested and studied on the basis of continuous problems, when encountering a practical MOCOP, practitioners tend to resort to popular MOEAs. Therefore, a relevant question is that between MOEAs and local search heuristics, which one may be more suitable for MOCOPs. In this paper, we attempt to answer this question. We choose seven well-known “baseline” MOEAs and local search heuristics in the area and systematically study their behaviours on four MOCOPs. We find that, unsurprisingly, different search paradigms have their own sweet spots; it depends on problem types, problem settings and search budgets. However, surprisingly, there exists one search heuristic, SEMO, which can be seen as a “transition” between the two search paradigms (i.e., a simple MOEA or randomised local search), that performs consistently better than the other heuristics across all the settings.

CCS CONCEPTS

• **Mathematics of computing** → **Combinatorics; Combinatoric problems**; • **Applied computing** → **Multi-criterion optimization and decision-making**.

KEYWORDS

Combinatorial optimisation, multi-objective optimisation, evolutionary algorithms, local search, empirical comparison

ACM Reference Format:

Miqing Li, Xiaofeng Han, Xiaochen Chu, and Zimin Liang. 2024. Empirical Comparison between MOEAs and Local Search on Multi-Objective Combinatorial Optimisation Problems. In *Genetic and Evolutionary Computation Conference (GECCO '24)*, July 14–18, 2024, Melbourne, VIC, Australia. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3638529.3654077>

1 INTRODUCTION

Multi-objective combinatorial optimisation problems (MOCOPs) represent a class of optimisation problems where decision variables are discrete in a finite search space and there is more than one objective needed to be optimised simultaneously. MOCOPs exist widely in various fields, ranging from software engineering [12] to machine learning [49], from logistics [3] to mechanical engineering [59], and from biology [28] to aerospace [23]. When encountering a practical MOCOP in their field, researchers and practitioners tend to resort to popular multi-objective evolutionary algorithms (MOEAs) to solve their problem, e.g., in [3, 12, 23, 28, 49, 59]. However, most of these MOEAs were developed, tested and studied on the basis of multi-objective continuous problems; and their performance may not be well verified on MOCOPs. In fact, discrete search spaces are rather distinct from continuous ones. MOEAs may behave very differently on these two types of optimisation problems [41].

On the other side, local search becomes increasingly popular in addressing MOCOPs within the communities of evolutionary computation and operational research [9]. Its systematic search manner works well with the neighbourhood construction in discrete spaces. Over the last two decades, there have been numerous studies navigating local search for MOCOPs. These studies include the design of efficient local search algorithms [5, 11, 33–35, 54, 60], analysis of its different components [19, 46, 55], exploration of its behaviours and algorithmic framework [17, 20, 51], automation of its design and algorithm selection [10, 47, 48], and utilisation for landscape analysis and visualisation [24, 45].

Now, an immediate question arises: between MOEAs and local search heuristics, which one is more suitable for MOCOPs, though there are observations that local search may perform better than MOEAs in some cases (e.g., in [56]). In this paper, we attempt to answer this question. We consider seven search heuristics. They include three well-known MOEAs, NSGA-II [15], SMS-EMOA [6]

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
GECCO '24, July 14–18, 2024, Melbourne, VIC, Australia
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0494-9/24/07.
<https://doi.org/10.1145/3638529.3654077>

and MOEA/D [66], representing three types of MOEAs, the Pareto-based, the indicator-based and the decomposition-based. For local search, we consider two well-established algorithms, Pareto local search (PLS) [54] and anytime Pareto local search (anytime PLS) [20]. PLS is one of the most well-known local search methods for MOCOPs [9]. Anytime PLS was developed to improve the anytime behaviour of PLS since before the completion, terminating PLS can lead to poor solution quality [20]. Moreover, we consider a very simple MOEA, SEMO [39], which is widely used in the theory community for running time analysis. SEMO can be seen as a “transition” between the two search paradigms (see Section II-B for detailed explanations). In addition, we also include random sampling as a baseline algorithm for comparison.

We consider four widely used MOCOPs, 0-1 knapsack [67], travelling salesman problem (TSP) [58], quadratic assignment problem (QAP) [37] and NK-landscape [2]. These problems are of various features, such as pseudo-boolean (0-1 knapsack and NK-landscape) and permutation (TSP and QAP), smooth (0-1 knapsack) and rugged (QAP and NK-landscape) landscapes, order-related permutation (TSP) and position-related permutation (QAP). For each problem, we consider two instances, small-scale and large-scale. Since different search paradigms (MOEAs and local search) may favour different budgets, for each problem instance we set two stop conditions, each favouring one search paradigm.

We find that, unsurprisingly, different search paradigms have their own sweet spots; it depends on problems, problem instances and search budgets. MOEAs are not certainly worse than local search on MOCOPs; actually, they are competitive on the majority of the problem settings considered, some even under the budget settings favouring local search. Probably most surprising results are from SEMO. SEMO, a simple MOEA that can be seen as a randomised local search (in analogy to single-objective case), performs consistently better than (or at least being competitive to) other MOEAs and local search heuristics across all the settings.

The rest of the paper is organised as follows. Section II introduces preliminaries of multi-objective optimisation and the search heuristics to be investigated. Section III describes experimental design, including the considered MOCOPs, performance metric, and parameter settings. Section IV gives experimental results. Lastly, Section V concludes the paper.

All the data is available at <https://github.com/Zim-L/MOEAvsLS>.

2 MULTI-OBJECTIVE OPTIMISATION

2.1 Definitions

Without loss of generality, let us consider a minimisation MOCOP with m objective functions $f(s) = (f_1(s), \dots, f_m(s))$ mapping a solution s in a finite decision space S to an objective space Z ($Z \subseteq \mathbb{R}^m$). Given two objective vectors $z, z' \in Z$, z is said to (Pareto) dominate z' (denoted by $z < z'$) iff for all $i \in \{1, \dots, m\}$, $z_i \leq z'_i$, and $z \neq z'$. For simplicity, one can extend the notion of Pareto dominance to solutions. That is, a solution s is said to dominate another solution s' iff $f(s) < f(s')$. Following [20], for a solution set A and a solution s , we say $A \not\prec s$ if there does not exist any solution $s' \in A$ such that $s' < s$ or $f(s') = f(s)$.

Based on the dominance relation, the notion of optimality in multi-objective optimisation can be defined. A solution $s \in S$ is

Algorithm 1 Random Sampling

```

1:  $A \leftarrow \emptyset$ 
2: repeat
3:    $s \leftarrow \text{random\_generation}()$  ▷ Generate a solution randomly in the search space
4:   if  $A \not\prec s$  then
5:      $A \leftarrow A \cup \{s\} \setminus \{a \in A \mid s < a\}$ 
6:   end if
7: until  $\text{stop\_condition}()$ 
Output:  $A$ 

```

Pareto optimal if there does not exist any $s' \in S$ such that $s' < s$. The set of Pareto optimal solutions is called the Pareto optimal set, and its mapping in the objective space is called the Pareto front. In many cases, finding the Pareto front can be computationally intractable. Therefore, the goal of search algorithms is to find a as good as possible approximation to the Pareto front so that the decision-maker can choose their most preferred solution from it.

2.2 Algorithms Investigated

In this work, we consider seven search heuristics. They are random sampling, Pareto local search (PLS) [54], anytime PLS [20], simple evolutionary multi-objective optimisation (SEMO) [39], and three popular MOEAs, NSGA-II [15], SMS-EMOA [6] and MOEA/D [66].

For each search heuristic, we care about the quality of its solutions generated during the search process. However, the final population of an MOEA cannot represent it [63] (see [42] for why this always happens). An MOEA's final population usually contains inferior solutions that are in fact dominated by the solutions discarded previously by the population update [43]. As such, we use an unbounded archive to store all nondominated solutions ever generated for the three MOEAs. This is also for random sampling. For PLS and SEMO, they already use an unbounded archive to store nondominated solutions for the exploration. However, for anytime PLS, its archive may not represent all nondominated solutions generated since it only preserves dominating solutions (usually at the beginning of the search), hence an extra archive being used as well.

Random Sampling. Random sampling, presented in Algorithm 1, is a search heuristic that draw solutions randomly from the search space. The archive A is used to store all nondominated solutions generated until the stop condition is met.

Pareto Local Search (PLS) [54]. Presented in Algorithm 2, PLS is one of the most well-known local search techniques for MOCOPs. It has been frequently studied in the area [4, 18, 31, 35, 46, 60]. PLS performs systematic exploration of solutions' neighbourhood. Using an archive to store all nondominated solutions found, PLS randomly selects one unexplored solution in the archive and explores all neighbours of the solution, while updating the archive by the generated neighbours. The algorithm halts either the stop condition is met or there is no unexplored solution in the archive.

Anytime Pareto Local Search (Anytime PLS) [20]. Shown in Algorithm 3, anytime PLS is an extension of PLS, aiming to improve the anytime behaviour of the original algorithm. It considers improvements on three components of a local search algorithm, selection step, acceptance criterion, and neighbourhood exploration. For the selection step, instead of choosing an unexplored solution in the archive randomly, anytime PLS considers the unexplored solution which has the best hypervolume contribution [20]. For

Algorithm 2 Pareto Local Search (PLS) [54]

```

1:  $s \leftarrow \text{rand\_generation}()$ 
2:  $\text{explored}(s) \leftarrow \text{FALSE}$ 
3:  $A \leftarrow \{s\}$ 
4: repeat
5:    $s \leftarrow \text{random\_selection}(\{a \in A \mid \text{explored}(a) = \text{FALSE}\})$   $\triangleright$  Select an unexplored
     solution randomly from  $A$ 
6:   for all  $s' \in \text{neighbourhood}(s)$  do  $\triangleright$  Neighbourhood exploration
7:     if  $A \not\ni s'$  then
8:        $\text{explored}(s') \leftarrow \text{FALSE}$ 
9:        $A \leftarrow A \cup \{s'\} \setminus \{a \in A \mid s' < a\}$ 
10:    end if
11:  end for
12:   $\text{explored}(s) \leftarrow \text{TRUE}$   $\triangleright$  Mark  $s$  being explored
13: until  $\text{stop\_condition}()$  or  $\{a \in A \mid \text{explored}(a) = \text{FALSE}\} = \emptyset$ 
Output:  $A$ 

```

Algorithm 3 Anytime PLS [20]

```

1:  $s \leftarrow \text{rand\_generation}()$ 
2:  $\text{explored}(s) \leftarrow \text{FALSE}$ 
3:  $A \leftarrow \{s\}; A' \leftarrow \{s\}$   $\triangleright$  Use  $A'$  to store nondominated solutions ever generated
4: repeat
5:    $s \leftarrow \text{OHI\_selection}(\{a \in A \mid \text{explored}(a) = \text{FALSE}\})$   $\triangleright$  Select an unexplored
     solution from  $A$  using the indicator optimistic hypervolume improvement (OHI) [20]
6:    $\text{dominating} \leftarrow \text{FALSE}$   $\triangleright$  Label for indicating current acceptance criterion
7:   for all  $s' \in \text{neighbourhood}(s)$  do
8:      $A' \leftarrow \text{archive\_update}(A', \{s'\})$ 
9:     if  $\exists a \in A$  such that  $s' < a$  then
10:       $\text{dominating} \leftarrow \text{TRUE}$   $\triangleright$  There exists at least one neighbour of  $s$  dominating a
        solution in  $A$ .
11:       $\text{explored}(s') \leftarrow \text{FALSE}$ 
12:       $A \leftarrow A \cup \{s'\} \setminus \{a \in A \mid s' < a\}$ 
13:    end if
14:  end for
15:  if  $\text{dominating} = \text{FALSE}$  then  $\triangleright$  Switch to accept nondominated solutions in the archive
16:    for all  $s' \in \text{neighbourhood}(s)$  do
17:      if  $A \not\ni s'$  then
18:         $\text{explored}(s') \leftarrow \text{FALSE}$ 
19:         $A \leftarrow A \cup \{s'\} \setminus \{a \in A \mid s' < a\}$ 
20:      end if
21:    end for
22:  end if
23:   $\text{explored}(s) \leftarrow \text{TRUE}$ 
24: until  $\text{stop\_condition}()$  or  $\{a \in A \mid \text{explored}(a) = \text{FALSE}\} = \emptyset$ 
Output:  $A'$ 

```

Algorithm 4 Simple Evolutionary Multiobjective Optimisation (SEMO) [39]

```

1:  $s \leftarrow \text{rand\_generation}()$ 
2:  $A \leftarrow \{s\}$ 
3: repeat
4:    $s \leftarrow \text{random\_selection}(A)$   $\triangleright$  Select a solution randomly from  $A$ 
5:    $s' \leftarrow \text{random\_selection}(\text{neighbourhood}(s))$   $\triangleright$  Select a neighbour of  $s$  randomly
6:   if  $A \not\ni s'$  then
7:      $A \leftarrow A \cup \{s'\} \setminus \{a \in A \mid s' < a\}$ 
8:   end if
9: until  $\text{stop\_condition}()$ 
Output:  $A$ 

```

the acceptance criterion, anytime PLS first considers solutions that dominate at least one solution in the archive, and if there is no such solution, then the algorithm switch to accept nondominated solutions as in PLS. For the neighbourhood exploration, there are two alternatives, first-improvement and best-improvement. Since it has shown that combining first-improvement with dominating acceptance criterion may be too greedy [20], we follow the suggestion to consider the best-improvement in the neighbourhood exploration.

Simple Evolutionary Multiobjective Optimisation (SEMO) [39].

As its name suggests, SEMO is a simple MOEA, and it has been frequently used in the theory community [7, 16, 27, 53, 57]. As shown in Algorithm 4, similar to PLS, SEMO randomly chooses one solution from the archive to perform neighbourhood search. However,

Algorithm 5 NSGA-II [15]

Input: n (population size)

```

1:  $A \leftarrow \emptyset$ 
2:  $P \leftarrow \text{initialisation}(n)$   $\triangleright$  Generate a population of  $n$  solutions randomly
3:  $A \leftarrow \text{archive\_update}(A, P)$   $\triangleright$  Update  $A$  by  $P$  based on dominance relation and ensure
   there is no duplicate nondominated solution in  $A$ 
4: repeat
5:    $Q \leftarrow \text{mating\_variation}(P, n)$   $\triangleright$  Generate  $n$  new solutions by mating selection and
     variation (e.g., crossover and mutation) on  $P$ 
6:    $A \leftarrow \text{archive\_update}(A, Q)$ 
7:    $(F_1, F_2, \dots, F_{\max}) \leftarrow \text{nondominated\_sorting}(P \cup Q)$   $\triangleright$  Partition all the solutions
     into different nondominated fronts
8:    $P \leftarrow \emptyset, i \leftarrow 1$ 
9:   while  $|P| + |F_i| \leq n$  do
10:      $\text{crowding\_distance}(F_i)$ 
11:      $P \leftarrow P \cup F_i$ 
12:      $i \leftarrow i + 1$ 
13:   end while
14:    $\text{sorting}(F_i)$   $\triangleright$  Sort solutions of the nondominated front  $F_i$  based on their crowding
     distances in a descending order
15:    $P \leftarrow P \cup F_i[1 : (n - |P|)]$   $\triangleright$  Place the first  $(n - |P|)$  solutions of  $F_i$  into  $P$ 
16: until  $\text{stop\_condition}()$ 
Output:  $A$ 

```

Algorithm 6 SMS-EMOA [6]

Input: n (population size)

```

1:  $A \leftarrow \emptyset$ 
2:  $P \leftarrow \text{initialisation}(n)$ 
3:  $A \leftarrow \text{archive\_update}(A, P)$ 
4: repeat
5:    $q \leftarrow \text{mating\_variation}(P, 1)$ 
6:    $A \leftarrow \text{archive\_update}(A, q)$ 
7:    $(F_1, F_2, \dots, F_{\max}) \leftarrow \text{nondominated\_sorting}(Q \cup q)$ 
8:    $s' \leftarrow \arg \min_{s \in F_{\max}} \text{hypervolume}(F_{\max} \setminus \{s\})$   $\triangleright$  Find the solution that has the least
     contribution to the hypervolume  $F_{\max}$ ; the tie is broken randomly
9:    $P \leftarrow P \cup \{q\} \setminus \{s'\}$ 
10: until  $\text{stop\_condition}()$ 
Output:  $A$ 

```

unlike PLS which considers all neighbours of the chosen solution, SEMO considers only one random neighbour, and hence can be seen as randomised Pareto local search. Note that the such an simple “1+1” MOEA with an unbounded archive was presented earlier in [25].

Non-dominated Sorting Genetic Algorithm II (NSGA-II) [15].

As presumably the most widely used MOEA, NSGA-II belongs to the Pareto-based algorithm class, characterised by non-dominated sorting and crowding distance procedures (Algorithm 5). In the selection process of NSGA-II, candidate solutions are partitioned into different non-dominated fronts. Solutions at lower fronts are regarded better than solutions at higher fronts. If solutions are at the same front, their crowding distance kicks in; the solutions with larger crowding distance are preferred.

S Metric Selection Evolutionary Multiobjective Optimisation Algorithm (SMS-EMOA) [6].

SMS-EMOA is a popular indicator-based MOEA. As can be seen in Algorithm 6, SMS-EMOA follows the non-dominated sorting procedure of NSGA-II, but instead of crowding distance, it uses the hypervolume indicator [67] to distinguish between solutions at the same nondominated front. That is, SMS-EMOA calculates the contribution of the hypervolume that each solution makes to the population, and removes solutions with the least contribution.

Multiobjective Evolutionary Algorithm Based on Decomposition (MOEA/D) [66]. MOEA/D is a well-known decomposition-based algorithm (Algorithm 7). It converts a multiobjective problem into a number of scalar optimisation subproblems by a set

Algorithm 7 MOEA/D [66]

Input: n (population size), $W = \{w_1, w_2, \dots, w_n\}$ (a set of n weight vectors), r (neighbourhood size)

```

1:  $A \leftarrow \emptyset$ 
2:  $P \leftarrow \text{initialisation}(n)$ 
3:  $z \leftarrow \text{refpoint\_calculation}(P)$   $\triangleright$  Calculate the reference point for the scalarising function
4:  $A \leftarrow \text{archive\_update}(A, P)$ 
5: repeat
6:   for  $i \leftarrow 1$  to  $n$  do
7:      $q \leftarrow \text{MOEAD\_variation}(P, r, w_i)$   $\triangleright$  Generate a solution based on the
       neighbourhood associated with weight vector  $w_i$  in  $P$ 
8:      $z \leftarrow \text{refpoint\_update}(q, z)$   $\triangleright$  Update the reference point
9:      $A \leftarrow \text{archive\_update}(A, q)$ 
10:     $P \leftarrow \text{population\_update}(P, q, r, w_i)$   $\triangleright$  Update the neighbourhood of the weight
       vector  $w_i$  based on the scalarising function
11:   end for
12: until  $\text{stop\_condition}()$ 
Output:  $A$ 

```

of well-distributed weight vectors and a scalarising function. In MOEA/D, neighbouring sub-problems work collaboratively, e.g., sharing the best solutions and performing recombination, which is a major difference from the other two classes of MOEAs.

3 EXPERIMENTAL DESIGN

3.1 Optimisation Problems Considered

Multi-Objective 0-1 Knapsack Problem (Knapsack). The multi-objective 0-1 knapsack problem [67] is a widely used MOCOP. Given a set of D items, each with m values and m weights, and a number of m knapsacks, each with a capacity (weight constraint), the knapsack problem can be defined as follows:

$$\begin{aligned}
 \max f_j(x) &= \sum_{i=1}^D v_{ji} x_i, \quad j = 1, \dots, m \\
 \text{subject to } \sum_{i=1}^D w_{ji} x_i &\leq c_j, \quad j = 1, \dots, m \\
 x &= (x_1, \dots, x_D) \in \{0, 1\}^D
 \end{aligned} \tag{1}$$

where $v_{ji} \geq 0$ is the value of item i in knapsack j , $w_{ji} \geq 0$ is the weight of item i in knapsack j , c_j is the capacity of knapsack j , and $x_i = 1$ means that item i is selected. Following the practice in [67], v_{ji} and w_{ji} are random integers in the interval $[10, 100]$, and the knapsack capacity was set to half of the total weight of all items for the knapsack. In the experiments, we consider two instances with different problem sizes (i.e., numbers of items): small-scale problem with $D = 100$ and large-scale problem with $D = 1000$.

Multi-Objective Travelling Salesman Problem (TSP). TSP is a classical combinatorial optimisation problem and it can be stated as follows. Given a number of cities and the cost of travel between each pair of them, the aim of the problem is to find the route that has the lowest travelling cost for visiting all the cities exactly once and returning to the starting point. The multi-objective TSP is an extended version of the standard TSP, in which there are multiple costs of travel between each pair of cities [58]. Formally, the problem can be defined as: given a network $L = (V, C)$, where $V = \{v_1, \dots, v_D\}$ is a set of D nodes and $C = \{C_j : j \in \{1, \dots, m\}\}$ is a set of m cost matrices between nodes ($C_j : V \times V$), the task is to find the Pareto optimal set of Hamiltonian cycles that minimise each of the m cost objectives. In our experiments, the m matrices are independent, generated by assigning each pair of cities with a

number randomly drawn from $[0, 1]$ [13]. Here, the experiments consider two instances with different problem sizes (i.e., numbers of cities): small-scale problem with $D = 50$ and large-scale problem with $D = 500$.

Multi-Objective Quadratic Assignment Problem (QAP). As a popular and widely used optimisation scenario, QAP refers to the problem of assigning a set of facilities to a set of locations with the admission of a flow (or cost) between facilities. The multi-objective QAP [37] is an extension of a basic QAP, where there are multiple flows between facilities, for example in hospital, one needs to consider the flows of doctors, patients, etc. Formally, a multi-objective QAP can be defined as: given $m \times D$ cost (flow) matrices $[C_{1ij}], \dots, [C_{mij}]$ and a $D \times D$ distance matrix $[L_{uv}]$, where C_{kij} is the k th cost between facilities i and j , and L_{uv} is the distance between locations u and v . A solution to the QAP is a permutation $x = (x_1, x_2, \dots, x_D)$ where x_i represents the location assigned to facility i . The objective functions can be formally defined as:

$$\min f_k(x) = \sum_{i=1}^D \sum_{j=1}^D C_{kij} L_{x_i x_j}, \quad k = 1, \dots, m \tag{2}$$

Following [37], the m cost matrices are independently generated, and each of their entries is randomly drawn from $[0, 100]$. For the distance matrix, according to [61], D locations are randomly generated in a 2-dimensional plane with the range of $[0, 5000]$, and Euclidean distance between locations are used as the distance. In our experiments, two problem instances with different sizes (i.e., numbers of facilities), small-scale problem with $D = 50$ and large-scale problem with $D = 200$, are considered.

Multi-Objective NK-Landscape Problem (NK-Landscape) NK-Landscape [2] is a commonly used problem in multi-objective optimisation due to its important feature that the ruggedness level of its problem landscape is tunable [64]. In the NK-landscape problem, N represents the number of bits (i.e., the number of decision variables, denoted by D here for consistency), and K represents the number of a bit's "neighbouring" bits that interact with it. Specifically, for each objective f_j , each bit x_i is associated with a contribution function $c_{ij}(x_i, x_{k_{ij1}}, \dots, x_{k_{ijK}})$, where index k_{ij1} denotes the first bit of its loci k_{ij} for the bit x_i on the j th objective, and so on. The problem is defined as follows:

$$\max f_j(x) = \frac{1}{D} \sum_{i=1}^D c_{ij}(x_i, x_{k_{ij1}}, x_{k_{ij2}}, \dots, x_{k_{ijK}}), \quad j = 1, \dots, m \tag{3}$$

Following the common practice [1, 14], c_{ij} was set to be a fitness contribution drawn randomly from $[0, 1]$ for every possible combination $\{0, 1\}^{(K+1)}$ of the loci k_{ij} . In our experiments, K was set to 10, and the small-scale NK-landscape problem has $D = 50$ and the large-scale one has $D = 200$.

3.2 Quality Indicator Used

Hypervolume (HV) [67] is a popular quality indicator, suitable for multi-objective optimisation problems where their Pareto front is unknown. HV calculates the volume of the objective space between a solution set and a reference point, and can reflect a comprehensive quality of the set in terms of convergence and diversity.

A critical issue in the calculation of HV is its reference point setting. If a solution does not fall into the area enclosed by the reference point, it will contribute zero to the HV value. It is advisable to set the reference point slightly worse (e.g., $\times 1.1$) than the nadir point of the problem's Pareto front. When the problem's Pareto front is unavailable, a common practice is to collect all nondominated solutions ever generated (by all the algorithms under all the settings), and then set the reference point based on the nadir point of the nondominated set.

However, this practice may not work in a situation where algorithms perform very differently; there may exist multiple algorithms that fall far behind good ones, hence having zero HV value. This is the case for all of the considered MOCOPs here. In an extreme case, all the algorithms except the best one receive a zero HV value. Therefore, in this paper, we consider a different approach. We use random sampling as a baseline to determine the reference point. Specifically, we consider the set of nondominated solutions ever generated by the random sampling under all the settings, and identify the nadir point of the set. Then, the reference point r is set to be slightly worse than the nadir point according to [40], namely, $r_i = \max_i + (\max_i - \min_i)/10$ for minimisation problems, and $r_i = \min_i - (\max_i - \min_i)/10$ for maximisation problems, where $\max_i$ and $\min_i$ denotes the maximum and minimum values of the nondominated set on the i th objective, respectively.

3.3 General Experimental Settings

Different search paradigms favour different settings. In general, local search may need more search budgets. Under very limited budgets, local search heuristics may only be able to explore very few solutions. MOEAs usually need fewer search budgets; they may quickly evolve forward but get trapped no matter how much more resources are added [41]. In our experiments, we consider two stop conditions. One is a budget of 100,000 evaluations, a common setting for MOEAs. The other is a budget of 1 hour wall clock time, a setting of favouring local search algorithms¹. It is worth mentioning that local search is usually quicker than MOEAs. In other words, with the same amount of time, local search can generate more solutions than MOEAs, i.e., more evaluations consumed. For local search, unlike MOEAs which usually consider the number of evaluations as the stop condition, it is a standard practice to use wall clock time.

We aim to consider appropriate search operators for specific combinatorial problems. For MOEAs, the uniform crossover [22] with a rate of 1.0 and the bit-flip mutation [22] with a rate of $1/D$ were used on the pseudo-boolean problems (i.e., Knapsack and NK-landscape), where D denotes the number of variables. TSP and QAP are two types of permutation problems, their solutions' representation having different meanings. TSP is based on order-related permutation while QAP is based on position-related permutation. Therefore, the order-based crossover and the 2-opt mutation (i.e., exchanging edges of the network) [22], which are suitable for order-based permutation, is used for TSP; and cycle-based crossover and the 2-swap mutation (i.e., exchanging locations of

the network) [22], suitable for position-based permutation, is used for QAP. For these two permutation problems, the crossover rate is 1.0 and the mutation rate is 0.05².

As for local search, the search operator is the bit flip for the pseudo-boolean problems. For TSP, the 2-opt operator is used, and for QAP, the 2-swap operator is used. Neighbours of a solution are defined as those that one bit-flip/2-opt/2-swap of the solution can reach on NK-landscape/TSP/QAP. However, for Knapsack, according to the definition of the problem, one bit-flip can lead to either improvement on both objectives or degradation on them, which is not reasonable and helpful for local search. Therefore, a neighbourhood within two bit-flips of a solution is used for Knapsack.

All parameters of the search heuristics are configured as the same as in their original papers. For the last three MOEAs, the population size is set to 100. For fair comparison, all the algorithms are written in Java and implemented on the JMetal platform [21]. For each problem instance, an algorithm is executed 30 runs, with each run on a core from Intel Xeon Gold 6248 with 2.50 GHz.

4 RESULTS

In this section, we present the comparative results amongst the seven algorithms based on the experimental design described in the previous section. HV results in the tables are the mean and standard deviation (SD) over 30 independent runs. To have statistically sound conclusions, we first used the Friedman rank-sum test [26] at a 0.05 significance level to examine the significance of the difference amongst all the algorithms. Friedman test is a nonparametric for comparison of multiple group data (more than two). We found these algorithms are significantly different on all the cases. Afterwards, we carried out Wilcoxon's rank-sum [29] test at a 0.05 significance level as a post-hoc test to compare all pairs of the algorithms, so that one knows for each algorithm, how it performs in comparison with the others. To avoid family-wise error in multiple comparisons, the Holm method [30] was applied to correct the p -values of Wilcoxon's rank-sum test.

4.1 On the Knapsack Problem

Table 1 gives the results of the seven algorithms on the Knapsack problem under the four settings, where the numbers in bold associated with each algorithm indicate the numbers of the comparisons that the algorithm performs statistically better, equally or worse than its competitor, at a 0.05 level by the Wilcoxon's rank-sum test (after Holm correction). Figure 1 plots the final archives of all the algorithms in a single run for the problem. This particular run, along with others for visual demonstration in the paper, is associated with the result which is the closest to the mean HV value.

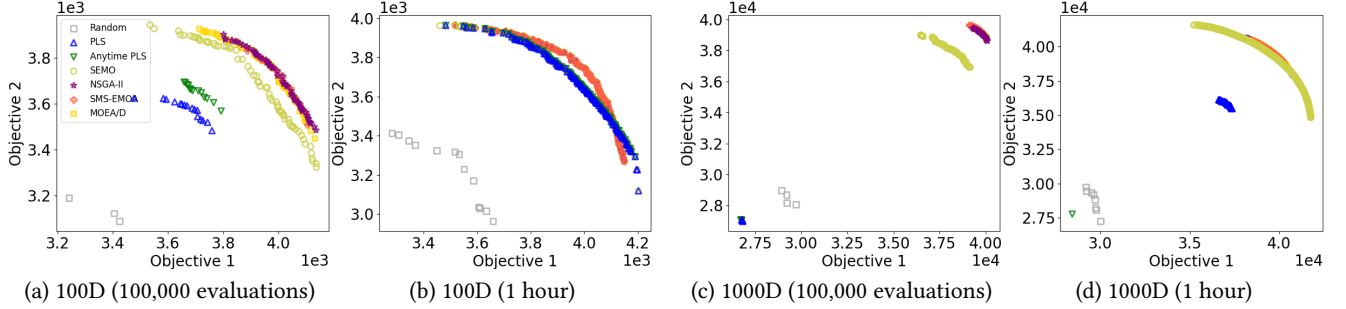
As can be seen from Table 1 and Figure 1, on the small-scale Knapsack under the small budget, local search generally performs worse than MOEAs. Between the two local search algorithms, anytime PLS outperforms PLS since the former prioritises dominating solutions, hence leading to better convergence. For the MOEAs, NSGA-II tends to perform best in terms of convergence, while SEMO can achieve much better coverage, hence a higher hypervolume

¹With the machine used, local search algorithms can explore a very large number of solutions. For example, on the Knapsack problem with 1000 items PLS can explore over 250,000,000 solutions on average.

²Setting the mutation rate to $1/D$ for permutation problems may be meaningless since the mutation is carried out based on the whole solution rather than its bits.

Table 1: HV results (mean and SD) of the seven algorithms as well as their statistical comparison on Knapsack under the four settings. The numbers in bold associated with each algorithm indicate the numbers of the comparisons that the algorithm performs statistically better, equally or worse than its competitor, respectively.

Problem	Budget	Random	PLS	Anytime PLS	SEMO	NSGA-II	SMS-EMOA	MOEA/D
Knapsack (100D)	100,000 evals	2.78e+4 (2.2e+4) 0/0/6	4.47e+5 (1.2e+5) 1/0/5	5.46e+5 (8.2e+4) 2/0/4	1.00e+6 (2.6e+4) 4/0/2	1.02e+6 (1.7e+4) 5/0/1	9.76e+5 (2.2e+4) 3/0/3	1.03e+6 (1.2e+4) 6/0/0
	1 hour	1.80e+5 (5.9e+4) 0/0/6	1.07e+6 (8.1e+3) 5/1/0	1.06e+6 (8.7e+3) 4/0/2	1.07e+6 (7.8e+3) 5/1/0	1.05e+6 (0.0e+0) 1/1/4	1.05e+6 (2.9e+1) 1/1/4	1.05e+6 (5.4e+2) 3/0/3
Knapsack (1000D)	100,000 evals	3.02e+5 (2.3e+5) 2/0/4	0.00e+0 (0.0e+0) 0/1/5	0.00e+0 (0.0e+0) 0/1/5	1.10e+8 (2.3e+6) 3/0/3	1.27e+8 (1.4e+6) 4/2/0	1.27e+8 (1.2e+6) 4/2/0	1.27e+8 (1.5e+6) 4/2/0
	1 hour	1.17e+6 (3.3e+5) 1/0/5	6.56e+7 (9.3e+6) 2/0/4	2.39e+4 (1.2e+5) 0/0/6	1.63e+8 (5.3e+5) 6/0/0	1.47e+8 (1.8e+6) 5/0/1	1.41e+8 (2.0e+6) 3/0/3	1.43e+8 (9.8e+5) 4/0/2
Total (better/equal/worse)		3/0/21	8/2/14	6/1/17	18/1/5	15/3/6	11/3/10	17/2/5

**Figure 1: The final archives of the seven algorithms in a single run on the Knapsack problem under the four settings (different dimensions and budgets). This particular run is associated with the result which is the closest to the mean HV value.**

value. When the algorithms are given sufficient computational resources (i.e., 1 hour), the advantage of local search emerges. PLS becomes one of the best algorithms, followed by anytime PLS. As for the three popular MOEAs, they perform fairly well in terms of convergence, but fail to spread their solutions, hence obtaining lower HV values than PLS and SEMO.

On the large-scale Knapsack (i.e., with 1000 items), SEMO shows clear advantage over the other algorithms under both small and large budgets. Under the small budget, interestingly, the local search algorithms are even worse than random sampling (Figure 1(c)). The reason for this occurrence is that with such a small budget available, local search can barely finish the exploration of very few solutions – the number of a solution’s neighbours is $\binom{1000}{2}$.

4.2 On the Travelling Salesman Problem

On TSP, MOEAs demonstrate large advantage over local search compared with on Knapsack. Table 2 gives the results of the seven algorithms under the four settings, and Figure 2 plots the final archives of all the algorithms in a single run.

On the small-scale problem under the small budget, SEMO is the best, followed by the three popular MOEAs and anytime PLS. Under the large budget, the three MOEAs obtain the best HV values, better than SEMO and the two local search algorithms. This is very different from the case on Knapsack, where under the large budget, SEMO and local search perform best. A possible explanation for this is that the landscape of TSP may have one or several big funnels (with numerous small attraction basins) [52, 65], where evolutionary algorithms can keep moving towards their bottom(s) gradually and slowly [65]. For the large-scale TSP, under the small budget, some MOEAs (e.g., MOEA/D) perform better than SEMO in terms of convergence, while under the large budget, SEMO has

much better convergence and spread. It is worth mentioning that anytime PLS performs best in terms of convergence, showing clear advantage over its original version PLS.

4.3 On the Quadratic Assignment Problem

On the QAP problem, the results are mixed, except for SEMO which consistently performs very best under the four settings, as shown in Table 3 and Figure 3. For the small-scale problem under the large budget, PLS performs better than the three MOEAs. For the small-scale problem under the small budget and the large-scale problem under the large budget, the MOEAs generally outperform PLS and anytime PLS, except on the large-scale case where anytime PLS is better than SMS-EMOA and MOEA/D. For the large-scale problem under the small budget, without any surprise, the local search heuristics struggle, with their solution quality worse than those obtained by random sampling.

4.4 On the NK-landscape Problem

NK-landscape (with $K = 10$) is the problem with the most rugged landscape out of all the considered problems. On such a problem, the three popular MOEAs struggle, as shown in Table 4 and Figure 4. Interestingly, once again, the simple evolutionary algorithm, SEMO, performs very well under all the settings. PLS and anytime PLS, comparable with SEMO, show clear advantage over the MOEAs, particularly on the large-scale problem. Random sampling, on such rugged landscape, may not be the worst choice. With sufficient budgets (1 hour), on the small-scale problem it can produce better solutions than MOEA/D.

Table 2: HV results (mean and SD) of the seven algorithms as well as their statistical comparison on the TSP problem under the four settings. The numbers in bold associated with each algorithm indicate the numbers of the comparisons that the algorithm performs statistically better, equally or worse than its competitor, respectively.

Problem	Budget	Random	PLS	Anytime PLS	SEMO	NSGA-II	SMS-EMOA	MOEA/D
TSP (50D)	100,000 evals	5.56e+1 (2.6e+0) 0/0/6	3.56e+2 (1.8e+1) 1/0/5	3.80e+2 (9.8e+0) 3/2/1	4.28e+2 (5.3e+0) 6/0/0	3.84e+2 (9.2e+0) 3/2/1	3.68e+2 (9.7e+0) 2/0/4	3.84e+2 (9.3e+0) 3/2/1
	1 hour	1.09e+2 (1.9e+0) 0/0/6	4.69e+2 (2.7e+0) 2/0/4	4.62e+2 (5.3e+0) 1/0/5	4.71e+2 (1.6e+0) 3/0/3	4.81e+2 (8.5e-1) 6/0/0	4.76e+2 (1.4e+0) 5/0/1	4.73e+2 (1.9e+0) 4/0/2
TSP (500D)	100,000 evals	7.85e+2 (3.6e+1) 2/0/4	3.66e+1 (4.7e+1) 0/1/5	6.28e+1 (8.0e+1) 0/1/5	2.27e+4 (6.3e+2) 5/0/1	1.98e+4 (1.1e+3) 3/0/3	2.12e+4 (1.0e+3) 4/0/2	2.38e+4 (7.1e+2) 6/0/0
	1 hour	1.27e+3 (4.4e+1) 0/0/6	4.59e+4 (2.2e+3) 1/0/5	5.12e+4 (1.1e+3) 4/1/1	5.71e+4 (8.0e+2) 6/0/0	4.99e+4 (1.1e+3) 3/0/3	4.77e+4 (1.4e+3) 2/0/4	5.08e+4 (9.4e+2) 4/1/1
Total (better/equal/worse)		2/0/22	4/1/19	8/4/12	20/0/4	15/2/7	13/0/11	17/3/4

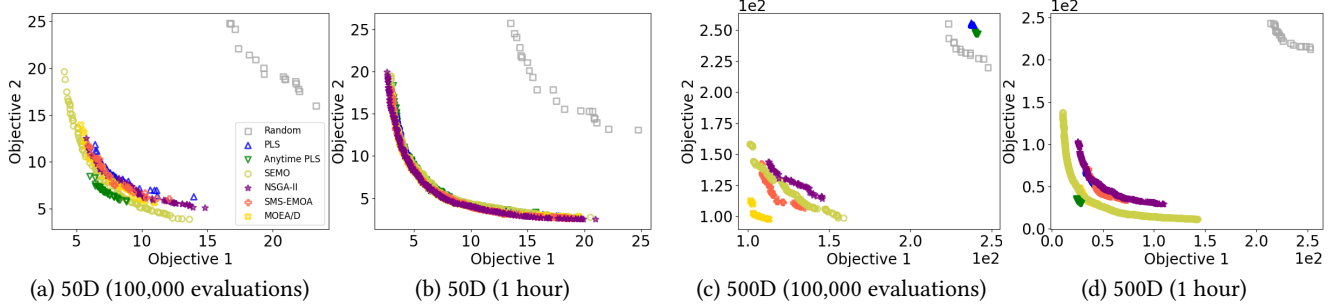


Figure 2: The final archives of the seven algorithms in a single run on the TSP problem under the four settings (different dimensions and budgets). This particular run is associated with the result which is the closest to the mean HV value.

Table 3: HV results (mean and SD) of the seven algorithms as well as their statistical comparison on the QAP problem under the four settings. The numbers in bold associated with each algorithm indicate the numbers of the comparisons that the algorithm performs statistically better, equally or worse than its competitor, respectively.

Problem	Budget	Random	PLS	Anytime PLS	SEMO	NSGA-II	SMS-EMOA	MOEA/D
QAP (50D)	100,000 evals	9.63e+13 (3.5e+12) 0/0/6	3.11e+14 (2.0e+13) 1/1/4	3.16e+14 (1.6e+13) 1/1/4	4.14e+14 (1.2e+13) 6/0/0	3.39e+14 (1.8e+13) 3/2/1	3.29e+14 (1.6e+13) 3/2/1	3.29e+14 (1.4e+13) 3/2/1
	1 hour	1.47e+14 (3.4e+12) 0/0/6	4.61e+14 (1.5e+13) 3/3/0	4.56e+14 (1.4e+13) 3/3/0	4.67e+14 (1.3e+13) 3/3/0	4.57e+14 (1.4e+13) 3/3/0	3.63e+14 (1.6e+13) 1/0/5	4.33e+14 (1.7e+13) 2/0/4
QAP (200D)	100,000 evals	3.58e+14 (5.4e+13) 2/0/4	1.67e+14 (1.8e+14) 0/1/5	2.15e+14 (2.0e+14) 0/1/5	1.28e+16 (4.5e+14) 6/0/0	9.20e+15 (5.4e+14) 3/2/1	9.16e+15 (5.0e+14) 3/2/1	9.13e+15 (5.4e+14) 3/2/1
	1 hour	4.91e+14 (4.6e+13) 0/0/6	1.22e+16 (9.7e+14) 1/0/5	1.84e+16 (6.5e+14) 4/0/2	2.48e+16 (3.9e+14) 6/0/0	2.07e+16 (8.5e+14) 5/0/1	1.39e+16 (5.7e+14) 2/0/4	1.58e+16 (6.6e+14) 3/0/3
Total (better/equal/worse)		2/0/22	5/5/14	8/5/11	21/3/0	14/7/3	9/4/11	11/4/9

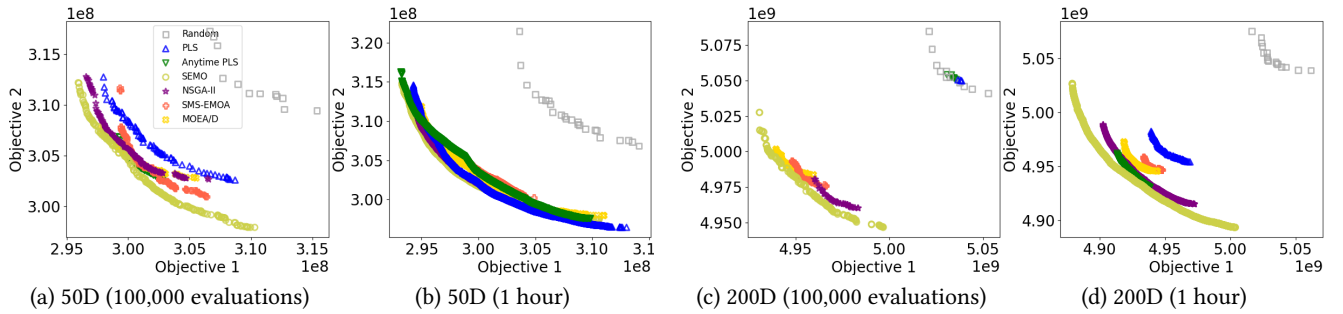


Figure 3: The final archives of the seven algorithms in a single run on the QAP problem under the four settings (different dimensions and budgets). This particular run is associated with the result which is the closest to the mean HV value.

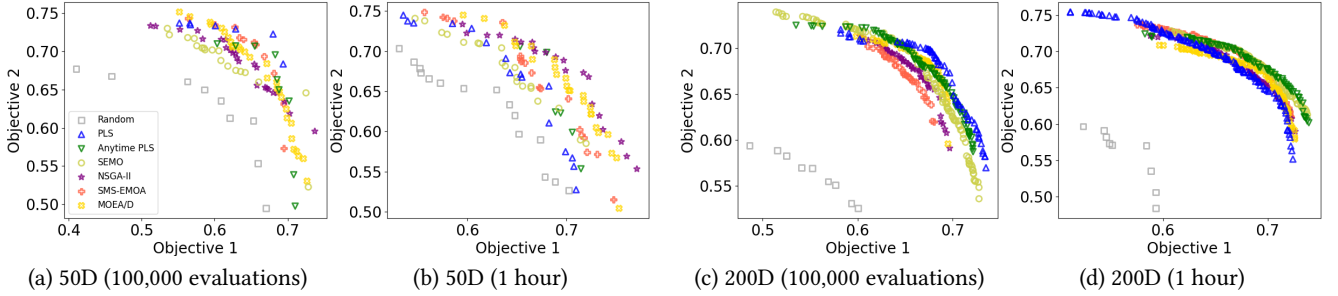
4.5 Summary

Table 5 summarises the results of the above pairwise statistical comparisons across all the four problems. As can be seen in the

table, local search and MOEAs have their own advantage, depending on problem size and budgets. On the small-scale problems under sufficient budgets, PLS perform very well, significantly better than the three popular MOEAs. In contrast, on the other cases the MOEA algorithms generally outperform PLS under the small budget, particularly on the large-scale problems. Between the two local

Table 4: HV results (mean and SD) of the seven algorithms as well as their statistical comparison on NK-Landscape under the four settings. The numbers in bold associated with each algorithm indicate the numbers of the comparisons that the algorithm performs statistically better, equally or worse than its competitor, respectively.

Problem	Budget	Random	PLS	Anytime PLS	SEMO	NSGA-II	SMS-EMOA	MOEA/D
NK-Landscape (50D)	100,000 evals	4.89e-2 (4.7e-3) 0/0/6	7.59e-2 (7.5e-3) 1/4/1	7.23e-2 (6.0e-3) 1/2/3	7.76e-2 (7.1e-3) 1/5/0	7.97e-2 (4.7e-3) 2/4/0	7.74e-2 (5.1e-3) 2/3/1	8.11e-2 (5.5e-3) 4/2/0
	1 hour	5.71e-2 (9.2e-3) 0/0/6	7.63e-2 (5.4e-3) 2/1/3	7.20e-2 (5.6e-3) 1/0/5	7.71e-2 (3.8e-3) 2/1/3	8.88e-2 (6.1e-3) 4/2/0	8.62e-2 (4.1e-3) 4/1/1	8.86e-2 (4.3e-3) 5/1/0
NK-Landscape (200D)	100,000 evals	4.76e-3 (9.4e-4) 0/0/6	4.23e-2 (1.6e-3) 4/1/1	4.27e-2 (2.6e-3) 4/1/1	4.41e-2 (1.5e-3) 6/0/0	3.61e-2 (2.6e-3) 1/1/4	3.41e-2 (3.0e-3) 1/1/4	3.87e-2 (2.0e-3) 3/0/3
	1 hour	5.30e-3 (1.4e-3) 0/0/6	4.56e-2 (2.2e-3) 4/1/1	4.30e-2 (3.2e-3) 1/2/3	4.70e-2 (1.9e-3) 4/2/0	4.74e-2 (2.6e-3) 5/1/0	4.13e-2 (3.3e-3) 1/1/4	4.34e-2 (2.6e-3) 2/1/3
Total (better/equal/worse)		0/0/24	11/7/6	7/5/12	13/8/3	12/8/4	8/6/10	14/4/6

**Figure 4: The final archives of the seven algorithms in a single run on the NK-landscape problem under the four settings (different dimensions and budgets). This particular run is associated with the result which is the closest to the mean HV value.****Table 5: Summary of pairwise statistical comparison on all the problems. The numbers associated with each algorithm indicate numbers of the comparisons that the algorithm performs statistically better, equally or worse than its competitor, respectively.**

Problem Size	Budget	Random	PLS	Anytime PLS	SEMO	NSGA-II	SMS-EMOA	MOEA/D
small	100,000 evals	0/0/24	4/5/15	7/5/12	17/5/2	13/8/3	10/5/9	16/6/2
	1 hour	0/0/24	12/5/7	9/3/12	13/5/6	14/6/4	11/2/11	14/1/9
large	100,000 evals	6/0/18	4/4/16	4/4/16	20/0/4	11/5/8	12/5/7	16/4/4
	1 hour	1/0/23	8/1/15	9/3/12	22/2/0	18/1/5	8/1/15	13/2/9
Total (better/equal/worse)		7/0/89	28/15/53	29/15/52	72/12/12	56/20/20	41/13/42	59/13/24

search heuristics, anytime PLS, which aims to improve the anytime behaviour of PLS, performs generally better than the original PLS under the small budgets, and is also competitive against MOEAs on many cases (e.g., small-scale with small budget). Amongst the three MOEAs, NSGA-II appears to be slightly better than SMS-EMOA and MOEA/D. This is different from observations on continuous problems, where SMS-EMOA and MOEA/D tend to obtain solutions with better quality on convergence.

A surprise comes from SEMO, a simple MOEA which can be seen as randomised local search. A possible explanation is its unbounded archive which is used to generate new solutions, in contrast to the bounded population in the three MOEAs. A possible reason that SEMO performs better than the two local search heuristics is its stochasticity mechanism, which may lead to less greedy search behaviour and help the search to jump out of local optima. In fact, similar conclusions have been derived analytically and empirically for MOEAs in recent studies [8, 44]. Particularly in [8], it has been reported that introducing stochasticity in population update can exponentially speed up the search of MOEAs.

5 CONCLUSIONS

This paper empirically compared six well-established MOEAs and local search heuristics, along with random sampling, on a range of

multi-objective combinatorial optimisation problems (MOCOPs). We found that in general MOEAs are competitive against local search, with the exception of on problems with highly rugged landscape (i.e., NK-landscape). Interestingly, the simple SEMO significantly outperforms the other “more advanced” MOEAs. This, on one side, suggests a need of new MOEAs for addressing MOCOPs, and on the other side, indicates the potential of evolutionary search on MOCOPs.

It is worth mentioning that we only consider the bi-objective problem instances with two different problem sizes and search budget settings (100,000 evaluations and 1 hour wall clock time). Other settings may lead to different conclusions; for example, higher budget may benefit local search heuristics on large-scale instances.

Note that the considered MOEAs and local search heuristics are rather “baseline”. Most of them were developed more than a decade ago. Sophisticatedly-designed search heuristics may result in different observations, such as local search techniques developed recently [11, 17, 35, 60]. Moreover, hybrid methods that combines local search with MOEAs seem to have high potential due to the fact that different search paradigms work better on different problems and settings, as shown in our experiments. This has already been well-confirmed by a number of studies in the area, e.g., [32–34, 36, 38, 50, 62].

REFERENCES

- [1] Hernán Aguirre and Kiyoshi Tanaka. 2007. Working principles, behavior, and performance of MOEAs on MNK-landscapes. *European Journal of Operational Research* 181, 3 (2007), 1670–1690.
- [2] Hernán E Aguirre and Kiyoshi Tanaka. 2004. Effects of elitism and population climbing on multiobjective MNK-landscapes. In *IEEE Congress on Evolutionary Computation*, Vol. 1. 449–456.
- [3] Fulya Altıparmak, Mitsuo Gen, Lin Lin, and Turan Paksoy. 2006. A genetic algorithm approach for multi-objective optimization of supply chain networks. *Computers & Industrial Engineering* 51, 1 (2006), 196–215.
- [4] Eric Angel, Evripidis Bampis, and Laurent Gourvés. 2004. Approximating the Pareto curve with local search for the bicriteria TSP (1, 2) problem. *Theoretical Computer Science* 310, 1-3 (2004), 135–146.
- [5] Matthieu Basseur, Rong-Qiang Zeng, and Jin-Kao Hao. 2012. Hypervolume-based multi-objective local search. *Neural Computing and Applications* 21 (2012), 1917–1929.
- [6] Nicola Beume, Boris Naujoks, and Michael Emmerich. 2007. SMS-EMOA: Multiobjective selection based on dominated hypervolume. *European Journal of Operational Research* 181, 3 (2007), 1653–1669.
- [7] Chao Bian, Chao Qian, and Ke Tang. 2018. A general approach to running time analysis of multi-objective evolutionary algorithms. In *International Joint Conference on Artificial Intelligence*. 1405–1411.
- [8] Chao Bian, Yawen Zhou, Miqing Li, and Chao Qian. 2023. Stochastic population update can provably be helpful in multi-objective evolutionary algorithms. In *the 32nd International Joint Conference on Artificial Intelligence*. 5513–5521.
- [9] Aymeric Blot, Marie-Éléonore Kessaci, and Laetitia Jourdan. 2018. Survey and unification of local search techniques in metaheuristics for multi-objective combinatorial optimisation. *Journal of Heuristics* 24, 6 (2018), 853–877.
- [10] Aymeric Blot, Marie-Éléonore Kessaci, Laetitia Jourdan, and Holger H Hoos. 2019. Automatic configuration of multi-objective local search algorithms for permutation problems. *Evolutionary Computation* 27, 1 (2019), 147–171.
- [11] Xinye Cai, Chao Xia, Qingfu Zhang, Zhiwei Mei, Han Hu, Lisong Wang, and Jun Hu. 2021. The collaborative local search based on dynamic-constrained decomposition with grids for combinatorial multiobjective optimization. *IEEE Transactions on Cybernetics* 51, 5 (2021), 2639–2650.
- [12] Tao Chen and Miqing Li. 2023. The weights can be harmful: Pareto search versus weighted search in multi-objective search-based software engineering. *ACM Transactions on Software Engineering and Methodology* 32, 1 (2023), 1–40.
- [13] David W. Corne and Joshua D. Knowles. 2007. Techniques for highly multiobjective optimisation: some nondominated points are better than others. In *Proceedings of the 9th Annual Conference on Genetic and Evolutionary Computation Conference (GECCO)*. 773–780.
- [14] Fabio Daolio, Arnaud Liefooghe, Sébastien Verel, Hernán Aguirre, and Kiyoshi Tanaka. 2015. Global vs local search on multi-objective NK-landscapes: contrasting the impact of problem features. In *Proceedings of the 2015 Annual Conference on Genetic and Evolutionary Computation*. 369–376.
- [15] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T Meyarivan. 2002. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation* 6, 2 (2002), 182–197.
- [16] Benjamin Doerr and Weijie Zheng. 2021. Theoretical analyses of multi-objective evolutionary algorithms on multi-modal objectives. In *AAAI Conference on Artificial Intelligence*. 12293–12301.
- [17] Miguel Ángel Domínguez-Ríos, Francisco Chicano, and Enrique Alba. 2021. Effective anytime algorithm for multiobjective combinatorial optimization problems. *Information Sciences* 565 (2021), 210–228.
- [18] Mădălina M Drugan and Dirk Thierens. 2012. Stochastic Pareto local search: Pareto neighbourhood exploration and perturbation strategies. *Journal of Heuristics* 18 (2012), 727–766.
- [19] Jérémie Dubois-Lacoste, Manuel López-Ibáñez, and Thomas Stützle. 2011. Improving the anytime behavior of two-phase local search. *Annals of Mathematics and Artificial Intelligence* 61, 2 (2011), 125–154.
- [20] Jérémie Dubois-Lacoste, Manuel López-Ibáñez, and Thomas Stützle. 2015. Anytime Pareto local search. *European Journal of Operational Research* 243, 2 (2015), 369–385.
- [21] Juan J. Durillo and Antonio J. Nebro. 2011. jMetal: A Java framework for multi-objective optimization. *Advances in Engineering Software* 42, 10 (2011), 760–771.
- [22] Agoston E Eiben and James E Smith. 2015. *Introduction to Evolutionary Computing*. Springer.
- [23] Matthew P Furrer and David B Spencer. 2006. Satellite constellation design tradeoffs using multiple-objective evolutionary computation. *Journal of Spacecraft and Rockets* 43, 6 (2006), 1404–1411.
- [24] Jonathan E Fieldsend and Khulood Alyahya. 2019. Visualising the landscape of multi-objective problems using local optima networks. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*. 1421–1429.
- [25] Jonathan E. Fieldsend, Richard M. Everson, and Sameer Singh. 2003. Using unconstrained elite archives for multiobjective optimization. *IEEE Transactions on Evolutionary Computation* 7, 3 (2003), 305–323.
- [26] Milton Friedman. 1940. A Comparison of Alternative Tests of Significance for the Problem of m Rankings. *The Annals of Mathematical Statistics* 11, 1 (1940), 86–92.
- [27] Oliver Giel and Per Kristian Lehre. 2006. On the effect of populations in evolutionary multi-objective optimization. In *Proceedings of the 8th Annual Conference on Genetic and Evolutionary Computation*. 651–658.
- [28] Julia Handl, Douglas B Kell, and Joshua Knowles. 2007. Multiobjective optimization in bioinformatics and computational biology. *IEEE/ACM Transactions on Computational Biology and Bioinformatics* 4, 2 (2007), 279–292.
- [29] Winston Haynes. 2013. Wilcoxon Rank Sum Test. In *Encyclopedia of Systems Biology*. Springer, New York, NY.
- [30] Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (1979).
- [31] Maarten Inja, Chiel Kooijman, Maarten de Waard, Diederik M Roijers, and Shimon Whiteson. 2014. Queued pareto local search for multi-objective optimization. In *Parallel Problem Solving from Nature—PPSN XIII: 13th International Conference*. Springer, 589–599.
- [32] Hisao Ishibuchi, Yasuhiro Hitotsuyanagi, Noritaka Tsukamoto, and Yusuke Nojima. 2009. Implementation of multiobjective memetic algorithms for combinatorial optimization problems: A knapsack problem case study. *Multi-Objective Memetic Algorithms* (2009), 27–49.
- [33] Hisao Ishibuchi and Tadahiko Murata. 1998. A multi-objective genetic local search algorithm and its application to flowshop scheduling. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)* 28, 3 (1998), 392–403.
- [34] Andrzej Jaszkiewicz. 2002. On the performance of multiple-objective genetic local search on the 0/1 knapsack problem - a comparative experiment. *IEEE Transactions on Evolutionary Computation* 6, 4 (2002), 402–412.
- [35] Andrzej Jaszkiewicz. 2018. Many-objective Pareto local search. *European Journal of Operational Research* 271, 3 (2018), 1001–1013.
- [36] Liangjun Ke, Qingfu Zhang, and Roberto Battiti. 2014. Hybridization of decomposition and local search for multiobjective optimization. *IEEE transactions on cybernetics* 44, 10 (2014), 1808–1820.
- [37] Joshua Knowles and David Corne. 2003. Instance generators and test suites for the multiobjective quadratic assignment problem. In *International Conference on Evolutionary Multi-Criterion Optimization*. Springer, 295–310.
- [38] Joshua D Knowles. 2002. *Local-search and hybrid evolutionary algorithms for Pareto optimization*. Ph. D. Dissertation. University of Reading UK.
- [39] M. Laumanns, L. Thiele, and E. Zitzler. 2004. Running time analysis of multiobjective evolutionary algorithms on pseudo-boolean functions. *IEEE Transactions on Evolutionary Computation* 8, 2 (2004), 170–182.
- [40] Miqing Li, Tao Chen, and Xin Yao. 2022. How to evaluate solutions in Pareto-based search-based software engineering? A critical review and methodological guidance. *IEEE Transactions on Software Engineering* 48, 5 (2022), 1771–1799.
- [41] Miqing Li, Xiaofeng Han, and Xiaochen Chu. 2023. MOEAs are stuck in a different area at a time. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*. 303–311.
- [42] Miqing Li, Manuel López-Ibáñez, and Xin Yao. 2023. Multi-objective archiving. *IEEE Transactions on Evolutionary Computation* (2023).
- [43] Miqing Li and Xin Yao. 2019. An empirical investigation of the optimality and monotonicity properties of multiobjective archiving methods. In *International Conference on Evolutionary Multi-Criterion Optimization*. Springer, 15–26.
- [44] Zimin Liang, Miqing Li, and Per Kristian Lehre. 2023. Non-elitist evolutionary multi-objective optimisation: Proof-of-principle results. In *Proceedings of the Companion Conference on Genetic and Evolutionary Computation*. 383–386.
- [45] Arnaud Liefooghe, Bilel Derbel, Sébastien Verel, Manuel López-Ibáñez, Hernán Aguirre, and Kiyoshi Tanaka. 2018. On Pareto local optimal solutions networks. In *International Conference on Parallel Problem Solving from Nature*. Springer, 232–244.
- [46] Arnaud Liefooghe, Jérémie Humeau, Salma Mesmoudi, Laetitia Jourdan, and El-Ghazali Talbi. 2012. On dominance-based multiobjective local search: design, implementation and experimental analysis on scheduling and traveling salesman problems. *Journal of Heuristics* 18 (2012), 317–352.
- [47] Arnaud Liefooghe, Gabriela Ochoa, Sébastien Verel, and Bilel Derbel. 2023. Pareto local optimal solutions networks with compression, enhanced visualization and expressiveness. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 713–721.
- [48] Arnaud Liefooghe, Sébastien Verel, Bilel Derbel, Hernán Aguirre, and Kiyoshi Tanaka. 2020. Dominance, indicator and decomposition based search for multi-objective QAP: landscape analysis and automated algorithm selection. In *International Conference on Parallel Problem Solving from Nature*. Springer, 33–47.
- [49] Zhichao Lu, Ian Whalen, Vishnu Boddeti, Yashesh Dhebar, Kalyanmoy Deb, Erik Goodman, and Wolfgang Banzhaf. 2019. NSGA-net: neural architecture search using multi-objective genetic algorithm. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 419–427.
- [50] Qizhang Luo, Guohua Wu, Bin Ji, Ling Wang, and Ponnuthurai Nagaratnam Suganthan. 2022. Hybrid multi-objective optimization approach with pareto local search for collaborative truck-drone routing problems considering flexible time

- windows. *IEEE Transactions on Intelligent Transportation Systems* 23, 8 (2022), 13011–13025.
- [51] Thibaut Lust and Jacques Teghem. 2010. Two-phase Pareto local search for the biobjective traveling salesman problem. *Journal of Heuristics* 16, 3 (2010), 475–510.
- [52] Gabriela Ochoa and Nadarajan Veerapen. 2018. Mapping the global structure of TSP fitness landscapes. *Journal of Heuristics* 24, 3 (2018), 265–294.
- [53] Edgar Covantes Osuna, Wanru Gao, Frank Neumann, and Dirk Sudholt. 2020. Design and analysis of diversity-based parent selection schemes for speeding up evolutionary multi-objective optimisation. *Theoretical Computer Science* 832 (2020), 123–142.
- [54] Luis Paquete, Marco Chiarandini, and Thomas Stützle. 2004. Pareto local optimum sets in the biobjective traveling salesman problem: An experimental study. In *Metaheuristics for Multiobjective Optimisation*. Springer, 177–199.
- [55] Luis Paquete and Thomas Stützle. 2009. Clusters of non-dominated solutions in multiobjective combinatorial optimization: An experimental analysis. In *Multi-objective Programming and Goal Programming*. Springer, 69–77.
- [56] Quan Minh Phan and Ngoc Hoang Luong. 2023. Pareto local search is competitive with evolutionary algorithms for multi-Objective neural architecture search. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 348–356.
- [57] Chao Qian, Yang Yu, and Zhi-Hua Zhou. 2013. An analysis on recombination in multi-objective evolutionary optimization. *Artificial Intelligence* 204 (2013), 99–119.
- [58] Celso C Ribeiro, Pierre Hansen, Pedro Castro Borges, and Michael Pilegaard Hansen. 2002. A study of global convexity for a multiple objective travelling salesman problem. *Essays and Surveys in Metaheuristics* (2002), 129–150.
- [59] R Saravanan, S Ramabalan, and C Balamurugan. 2009. Evolutionary multi-criteria trajectory modeling of industrial robots in the presence of obstacles. *Engineering Applications of Artificial Intelligence* 22, 2 (2009), 329–342.
- [60] Jialong Shi, Jianyong Sun, Qingfu Zhang, Haotian Zhang, and Ye Fan. 2022. Improving Pareto local search using cooperative parallelism strategies for multiobjective combinatorial optimization. *IEEE Transactions on Cybernetics* (2022).
- [61] Eric D Taillard. 1995. Comparison of iterative searches for the quadratic assignment problem. *Location Science* 3, 2 (1995), 87–105.
- [62] El-Ghazali Talbi. 2015. Hybrid metaheuristics for multi-objective optimization. *Journal of Algorithms & Computational Technology* 9, 1 (2015), 41–63.
- [63] Ryoji Tanabe and Akira Oyama. 2017. Benchmarking MOEAs for multi-and many-objective optimization using an unbounded external archive. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 633–640.
- [64] Sébastien Verel, Arnaud Liefoghe, Laetitia Jourdan, and Clarisse Dhaenens. 2013. On the structure of multiobjective combinatorial search space: MNK-landscapes with correlated objectives. *European Journal of Operational Research* 227, 2 (2013), 331–342.
- [65] Darrell Whitley, Doug Hains, and Adele Howe. 2010. A hybrid genetic algorithm for the traveling salesman problem using generalized partition crossover. In *Parallel Problem Solving from Nature, PPSN XI: 11th International Conference, Kraków, Poland, September 11-15, 2010, Proceedings, Part I* 11. Springer, 566–575.
- [66] Qingfu Zhang and Hui Li. 2007. MOEA/D: A multiobjective evolutionary algorithm based on decomposition. *IEEE Transactions on Evolutionary Computation* 11, 6 (2007), 712–731.
- [67] Eckart Zitzler and Lothar Thiele. 1999. Multiobjective evolutionary algorithms: A comparative case study and the strength Pareto approach. *IEEE Transactions on Evolutionary Computation* 3, 4 (1999), 257–271.